02207 Verification of Digital Systems Assignment

SyoSil Aps ©

2026

Contents

1 Introduction **3**

2 Memory Arbiter Design Specification **3**
2.1 APB Control 5
2.2 Priority Selector 7
2.2.1 Sorting State Machine 7
2.2.2 Priority Switch Logic 8
2.3 CRC Module 9
2.4 Design Requirements 10

3 SyoSil Data Transfer Protocol **11**

4 Simple SyoSil Data Transfer Protocol **12**

5 Memory Arbiter Testbench **13**

6 Assignment **15**
6.1 A1. Development of a Verification Plan 15
6.2 A2. Integration of *uVC*s in the testbench 15
6.3 A3. Coverage 16
6.4 A4. Fully random testcase 18
6.5 A5. *sSDT* and *SDT* Protocol Checkers 18
6.6 A6. Reference Model and Scoreboard 18
6.7 A7. *Memory Arbiter* Timing Checker 20
6.8 A8. Additional Tasks 20

1 Introduction

The following document provides:

1. Documentation of the *Memory Arbiter* (MARB) and the used protocols.
2. The tasks for the final assignment.

2 Memory Arbiter Design Specification

A *Memory Arbiter* is a device used in a shared memory system to determine which client will be allowed to access the shared memory, ensuring proper coordination and preventing conflicts.

Figure 2.1, shows a high level diagram of the provided *Memory Arbiter* IP. The IP has the following external interfaces/signals:

- `Clock`
- `Reset` (asynchronous)
- `Client interface #1` (CIF1 - SDT protocol)
- `Client interface #2` (CIF2 - SDT protocol)
- `Client interface #3` (CIF3 - SDT protocol)
- `CRC interface` (CRC - SSDT protocol)
- `Advanced Peripheral Bus` (APB protocol)
- `Memory interface` (MIF - SDT protocol)

As can be seen in Figure 2.1, the device uses three data transfer protocols.

- The SyoSil Data Transfer Protocol (*SDT*) is used by the external clients to make requests to access the memory and by the *Memory Arbiter* to access the memory.
- The Simple SyoSil Data Transfer Protocol (*sSDT*) is used by the CRC module to broadcast the CRC8 checksum of the current MIF transaction.
- The Advanced Peripheral Bus (APB) is used to configure the device by writing its registers.

The core logic of the *Memory Arbiter* arbitrates between the three client interfaces (CIFs) to access the memory for `read` and `write` operations and always selects the CIF with the highest priority to be served. The IP always checks for the latest priority configuration. The priority configuration changes, the IP changes the order of the client requests as long as there is no client being served at that moment. When a request has been selected the *Memory Arbiter* drives the MIF with the request. Then, the MIF enables the `ack` signal to close the handshake.

The *Memory Arbiter* also generates the CRC8 checksum of the current MIF transaction using the standard System Management Bus (SMBus) polynomial:

$$x^8 + x^2 + x + 1$$

The checksum is calculated on the concatenated address and data value.

The *Memory Arbiter* contains two registers that allows to manage the arbitration of the access from the clients: `Control Register` and `Dynamic Priority Register` and they are physically located in the submodule `APB Control`.

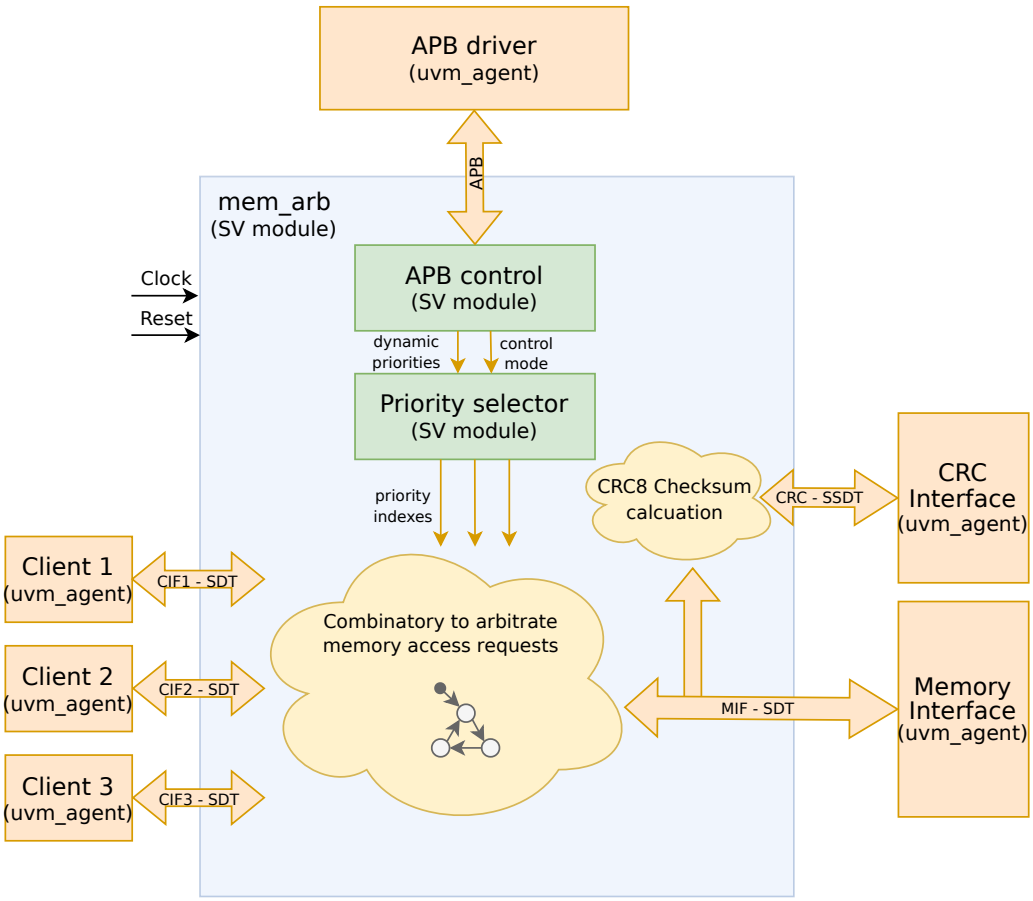


Fig. 2.1: High level diagram of the Memory Arbiter design.

2.1 APB Control

This module is designed to interface with an APB driver through an APB bus and `write` and `read` registers.

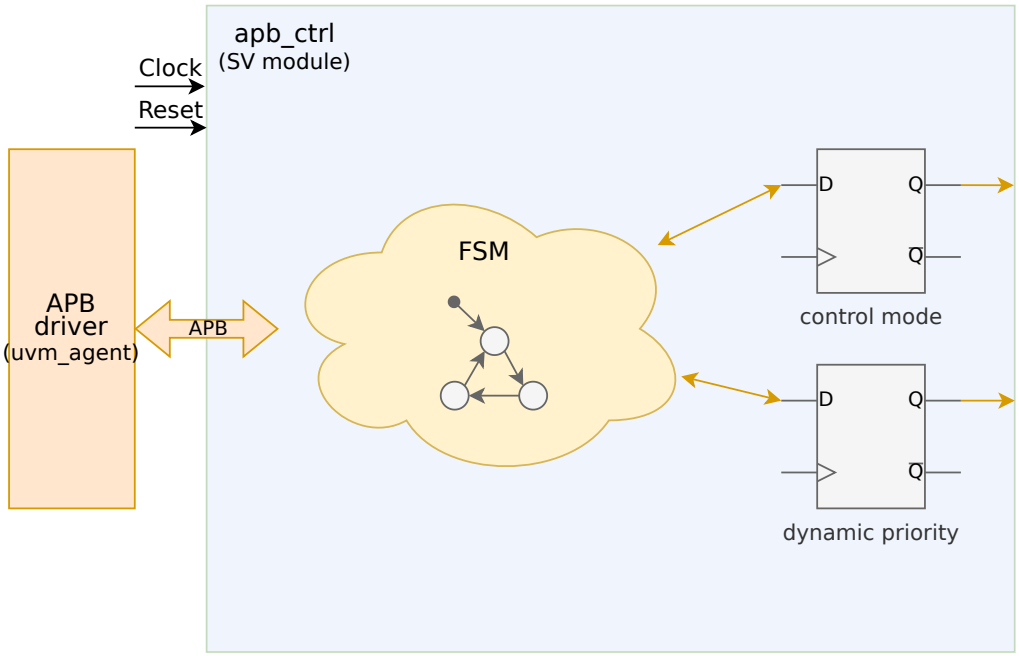


Fig. 2.2: Diagram of the APB Control module.

Fig 2.2 shows the block diagram of the APB Control module. When a `write` or `read` request arrives from the interface, the APB control module processes this request acting on the internal registers.

The APB (Advanced Peripheral Bus) is a part of the AMBA (Advanced Microcontroller Bus Architecture) protocol family, designed for low-power and low-bandwidth communication with peripheral devices. An APB slave is a peripheral device that responds to read and write operations initiated by an APB master. The APB slave performs specific functions such as reading from or writing to registers, and it communicates the status of these operations back to the master.

The APB control module contains two registers that allow the user to configure the dynamic priority of the CIFs and the arbitration mode.

The `control` register is used to manage the arbitration mode. The field `enable` is used to enable or disable the *Memory Arbiter*. If the *Memory Arbiter* is enabled, then the field `mode` can be used to change the arbitration mode between `static` (`mode == 0`) or `dynamic` (`mode == 1`) prioritization of the CIFs. The `dynamic` prioritization of the CIFs is set by the `Dynamic Priority` register, that contains a field for each CIF (the field for Client 4 is unused by the DUT), where the priority value can be set.

As shown in Table 2.1, the `Dynamic priority reg` contains a field for each CIF, where the priority value can be set. Each field is 8 bits wide, allowing for 256 priority levels. The higher the value, the higher the priority the client has. The default value is set to 0. The `Dynamic priority reg` may only be changed while the *Memory Arbiter* is disabled e.g. `enable` is set to 0. After changing the value of the `Dynamic priority reg` the *Memory Arbiter* has to update it's internal priority. This takes 6 clock cycles, which has to be accounted for when

Register	Address	Fields			
Dynamic Priority	0x04	Unused [31:24]	Client 3 [23:16]	Client 2 [15:8]	Client 1 [7:0]
Control mode	0x00		Unused [31:3]	Mode [2:1]	Enable [0]

Table 2.1: Structure of the APB registers.

changing priorities. The internal priorities are only updated when the *Memory Arbiter* is in `dynamic` mode. If the priority register is updated while the *Memory Arbiter* is in `static` mode the delay for updating the internal priorities occurs when changing the mode field to `dynamic`.

The APB interface signals are as follows:

Signal name	Producer direction	Consumer direction	Comment
`conf_wr`	output	input	This signal indicates a write operation when asserted.
`conf_sel`	output	input	This signal selects the peripheral for the current transaction.
`conf_enable`	output	input	This signal is intended to enable the transaction. (Unused in this design).
`conf_addr`	output	input	This 32-bit signal specifies the address for the current transaction.
`conf_wdata`	output	input	This 32-bit signal carries the data to be written during a write transaction.
`conf_strb`	output	input	This 4-bit signal is the write strobe, indicating which bytes of `conf_wdata` are valid.
`conf_rdata`	input	output	This 32-bit signal carries the data read from the peripheral during a read transaction.
`conf_ready`	input	output	This signal indicates that the peripheral is ready to complete the current transaction.
`conf_slverr`	input	output	This signal indicates that the peripheral is ready to complete the current transaction.

Table 2.2: Signals of APB interface.

2.2 Priority Selector

The `Priority Selector` module is designed to select the client priorities based on the `Control Mode` and the `Dynamic Priority` signals. To this aim, it instantiates and controls a cascade of single sorter modules to sort the clients based on their priorities.

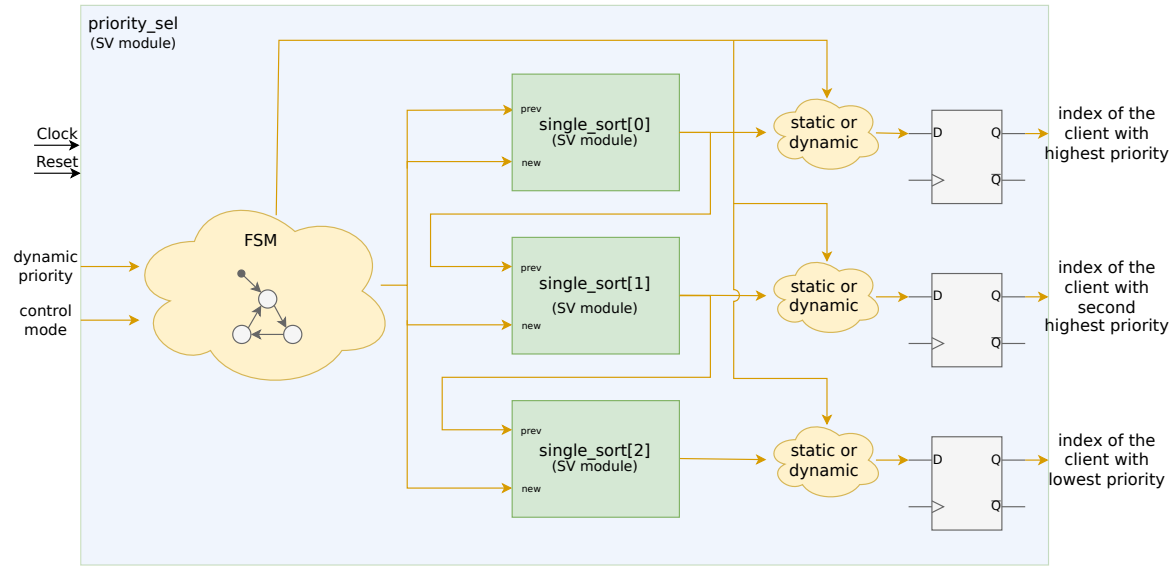


Fig. 2.3: Diagram of the Priority Selector design.

Fig 2.3 shows the architecture of the `priority_sel` module. It supports both static and dynamic priority selection based on the `Control Mode` signal. In static mode, the priority list is fixed, and the priority order is:

- `CIF#1` before `CIF#2`
- `CIF#1` before `CIF#3`
- `CIF#2` before `CIF#3`

In dynamic mode, the priority list is updated based on the `Dynamic Priority` signal values when the sorting state machine is idle. The sorting algorithm takes as many clock cycles as the number of instantiated clients to complete the sorting process.

2.2.1 Sorting State Machine

As shown in Fig 2.4, the sorting state machine has three states: `SORTER_IDLE`, `SORTER_CLEAR`, and `SORTER_SORT`.

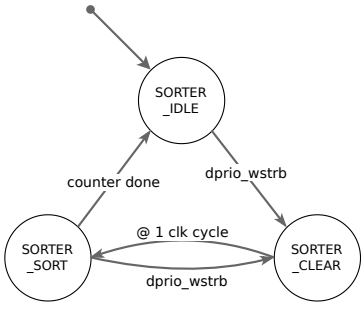


Fig. 2.4: Diagram of the Priority Selector FSM.

- In `SORTER_IDLE`, the sorters hold their values and wait for the next sorting cycle.
- In `SORTER_CLEAR`, the sorters are cleared from previous content to prepare for a new sorting operation.
- In `SORTER_SORT`, the sorting process is enabled and the clients are sorted based on their priorities. Using this state, the sorting algorithm iterates through the clients, comparing and swapping their priorities as needed until the list is fully sorted.

The state machine transitions among these states based on the control signals and the current state of the sorting process. The sorting process ensures that the module provides to the *Memory Arbiter* an array where the highest priority client index is in position 0, the second highest in position 1 and so on.

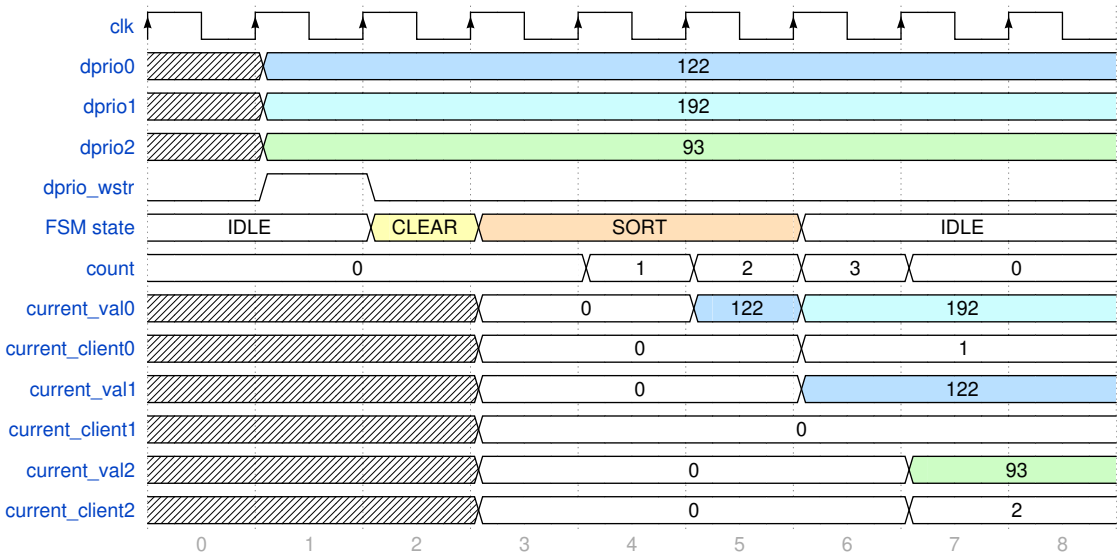


Fig. 2.5: Time evolution of sorting algorithm signals.

Fig 2.5 illustrates the time evolution of the signals involved in the sorting algorithm. The event that triggers the FSM is the pulse on the `dprio_wstrb` signal (meaning at least a new priority value has been written in the registers). The FSM clears all the flip-flops in the single sorter submodules before starting the sorting sequence. During the sorting sequence, the counter is increased until all the clients are processed. In particular, when the counter:

- is equal to `0` the reg `dprio0` is taken into account
- is equal to `1` the reg `dprio1` is taken into account
- is equal to `2` the reg `dprio2` is taken into account

At the end of the sorting algorithm, the value of `current_client` signals are respectively 1, 0 and 2 because the client with the highest priority is the one with index 1 (`dprio1 == 192`), the client with second highest priority is the one with index 0 (`dprio0 == 122`) and the client with lowest priority is the one with index 2 (`dprio0 == 122`)

2.2.2 Priority Switch Logic

The priority switch logic switches between static and dynamic priority according to the operating mode (`Control Mode`). In dynamic mode, the priority list is only updated when the sorting state machine is `SORTER_IDLE`. The priority list holds the clients with the highest priority at index 0, second highest at index 1, and so on.

2.3 CRC Module

The CRC Module calculates the checksum of the current transaction on the MIF. The checksum is calculated on the combined address and data bitvector:

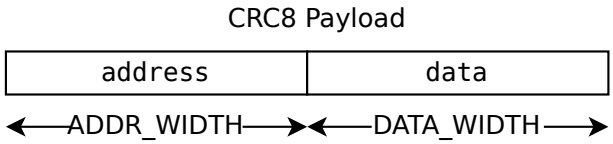


Fig. 2.6: Payload of the CRC8 module

The checksum is calculated as CRC8 using the SMBus polynomial:

$$x^8 + x^2 + x + 1$$

and broadcast using the *sSDT* interface. The checksum is always broadcast when the MIF acknowledges a transaction.

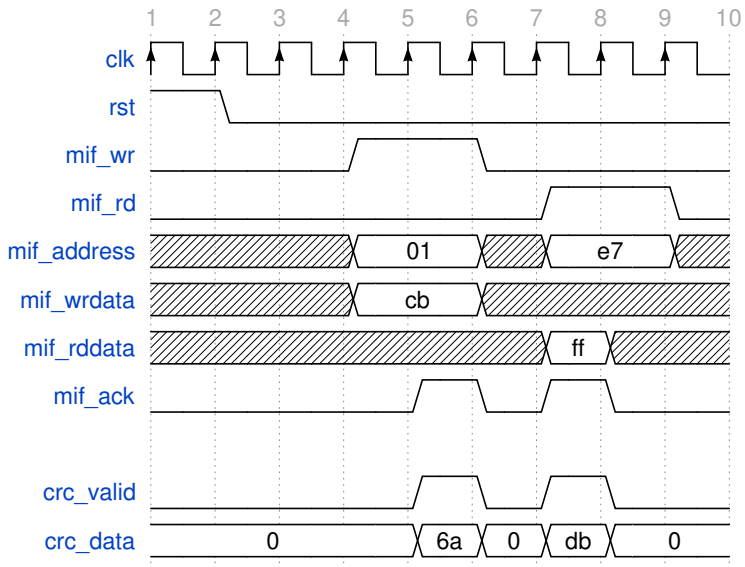


Fig. 2.7: CRC Interface timing

2.4 Design Requirements

The *Memory Arbiter* operation respects the following requirements:

- **DR01:** It shall be possible to reset the state of the *Memory Arbiter* by toggling the `rst` input signal;
- **DR02:** All signals shall react to the rising edge of the input `clk` signal;
- **DR03:** The architecture contains: 3 CIFs, 1 MIF, 1 APB;
- **DR04:** The priority of the clients is configurable by registers;
- **DR05:** The priority of the clients can change dynamically;
- **DR06:** The *Memory Arbiter* must be disabled to change the dynamic priority;
- **DR07:** Before re-enabling the *Memory Arbiter* after a dynamic priority change, the internal sort algorithm must be finished.
- **DR08:** The MIF should communicate with a single CIF at a time. Two (or more) CIFs cannot be ACK'ed in the same clock cycle;
- **DR09:** The MIF should accept the request of the CIF with the highest priority. If all clients have the same priority then the default priority must be respected: CIF1 before CIF2 before CIF3;
- **DR10:** The MIF should close handshake/communication with CIF by sending an acknowledge signal;
- **DR11:** The CIF and the MIF are following *SDT* protocol.
- **DR12:** The CRC interface follows the *sSDT* protocol.
- **DR13:** The CRC interface broadcasts the checksum in the acknowledge phase of the MIF.
- **DR14:** The CRC module calculates the checksum using the polynomial `0x07` based on the vector `{addr,data}`.

3 SyoSil Data Transfer Protocol

The SyoSil Data Transfer Protocol (*SDT*) is a traditional data transfer protocol. The protocol contains 6 signals: `rd`, `wr`, `addr`, `rd_data`, `wr_data` and `ack`. Based on the direction of the signals the protocol can be used in two variants: as a `producer` and as a `consumer`. The Table 3.3 describes the two variants.

Signal name	Producer direction	Consumer direction	Comment
rd	output	input	When asserted, the client requests a read.
wr	output	input	When asserted, the client requests a write.
addr	output	input	The read/write address. Valid when `rd` or `wr` is asserted.
rd_data	input	output	The read data. Valid when `ack` is asserted.
wr_data	output	input	The write data. Valid when `wr` is asserted
ack	input	output	Acknowledge signal. DUT acknowledges when a request has been served.

Table 3.3: SDT protocol

A timing diagram outlining the behavior of the protocol is shown in the Figure 6.1.

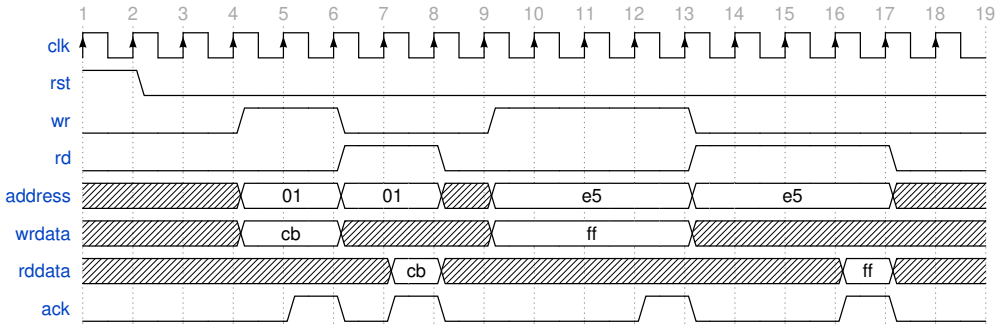


Fig. 3.1: Write and read operations using the *SDT* protocol.

The protocol has the following parameters:

- `ADDR_WIDTH`, the width of the address signal
- `DATA_WIDTH`, the width of the data signal

The SDT protocol has the following invariants:

- Asserting `rd` and `wr` at the same time is illegal
- When `rd` or `wr` is asserted, then addr must not be `X`
- When `wr` is asserted, then wr_data must not be `X`

4 Simple SyoSil Data Transfer Protocol

The simple SyoSil Data Transfer (*sSDT*) protocol is a simple synchronous data transfer protocol. The protocol has two signals, data and valid, and has two variants, the "producer" and the "consumer", which are modifying the direction of the signal as showed in the Table 4.4.

Signal Name	Producer direction	Consumer direction	Comment
valid	output	input	when asserted, data is valid
data	output	input	the data of the protocol

Table 4.4: *sSDT* protocol signals.

The protocol operation must follow these protocol requirements (PR):

1. **Reset requirement (PR1)**: when `rst` is high, `valid` can not be high
2. **Data validity requirement (PR2)**: when `valid` is high, `data` must have a valid value
3. **Data invalidity requirement (PR3)**: when `valid` is low, `data` must be `0`

A timing diagram outlining the behavior of the protocol is shown in Figure 4.1. All requirements should be valid when checked synchronously to the clk.

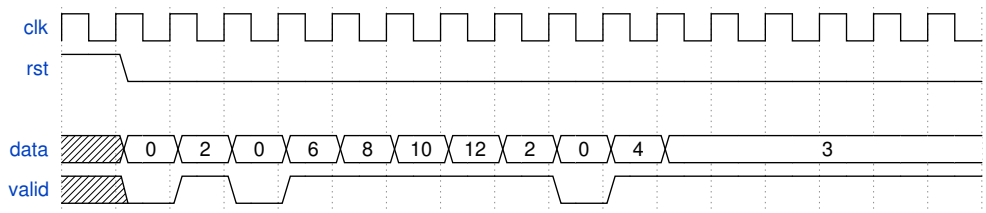


Fig. 4.1: *sSDT* protocol waveform.

5 Memory Arbiter Testbench

For this assignment an incomplete implementation of the testbench is provided in `assignment/marb/src`. Figure 5.1 shows a complete testbench with the missing components and connections marked in yellow

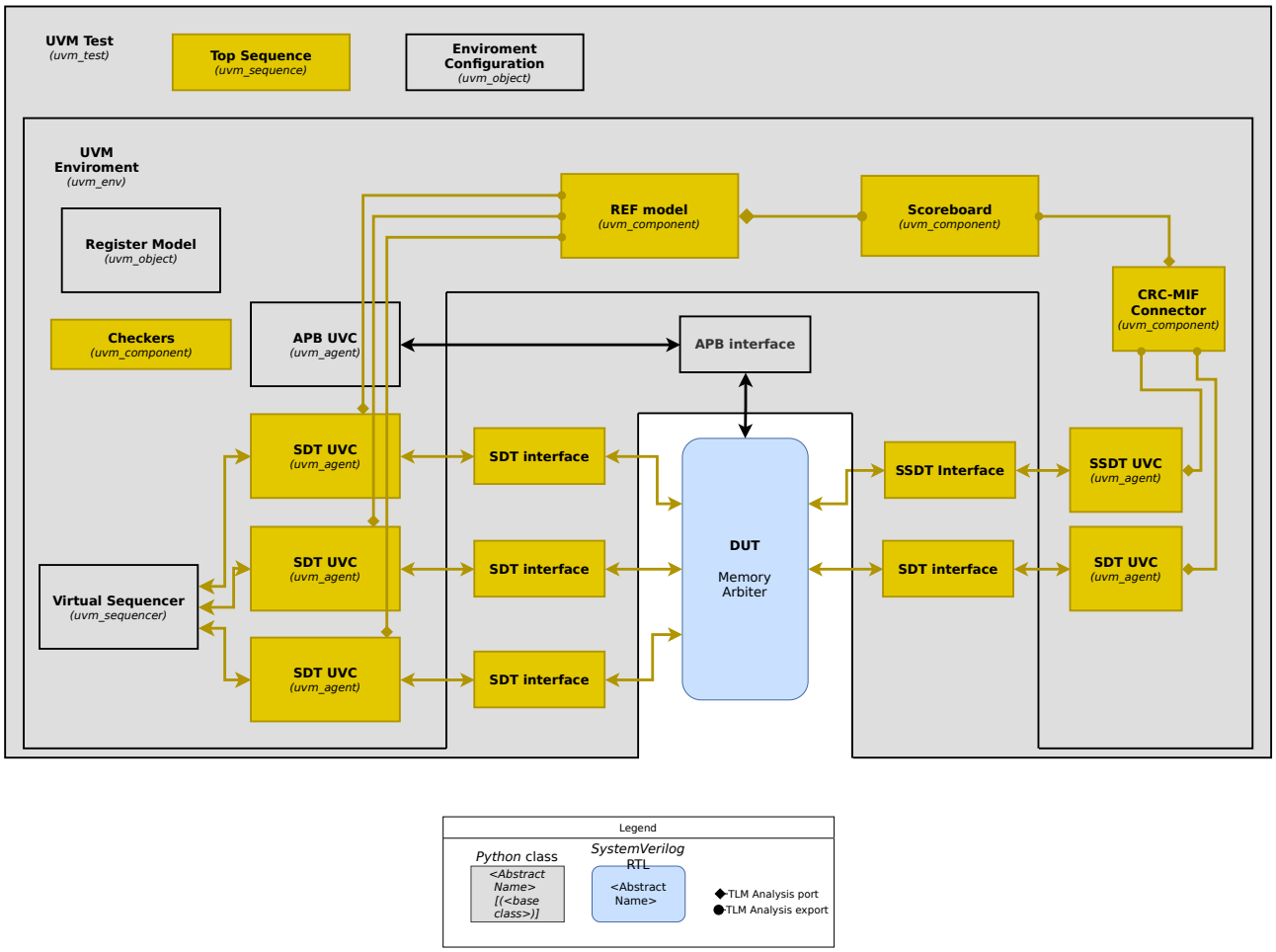


Fig. 5.1: Memory Arbiter testbench

The management of the arbitration for the clients connected to the *Memory Arbiter* is dependent on writing to registers. To handle reading from and writing to registers the virtual sequencer contains four functions:

- `write_ctrl_reg(enabled, mode)`
- `read_ctrl_reg()`
- `write_dprio_reg(prio0, prio1, prio2)`
- `read_dprio_reg()`

The write functions write the provided data into the registers. The read functions will return the register content as a 32bit integer.

The functions also mirror the register values into the `cl_marb_reg_model` object. The register model object can be used if knowledge of current register status is necessary anywhere in the testbench.

The DUT supports parametrization of the *SDT* interfaces with the parameters `ADDR_WIDTH`

and `DATA_WIDTH`. The default value for both is 8. For this assignment we will disregard the parameters until A8. The *SDT uVC*s default values match the DUTs default value. In other cases where `ADDR_WIDTH` or `DATA_WIDTH` are needed you can hardcode the values.

6 Assignment

This section describes tasks you need to complete for the assignments.

The assignment covers the following tasks:

A1. Development of a Verification Plan;

A2. Integration of *uVC*s in the testbench;

A3. Implementation of coverage;

A4. Implementation of randomized test cases;

A5. Implementation of checkers for the *SDT* protocol;

A6. Implementation and integration of the Reference Model and Scoreboard;

A7. Implementation of checkers for the *Memory Arbiter*;

A8. Bonus tasks.

6.1 A1. Development of a Verification Plan

Lecture references:

1. Lecture08, LE14, E12

Read the provided documentation and use `assignment/template_svp.csv` to create the verification plan. Each entry into the verification plan should specify:

1. The targeted design requirement
2. The coverage to collect, if applicable.
3. The verification criteria, i.e. how is the requirement checked.
4. The verification priority.
5. Type, i.e. where is this requirement coming from (Design/Protocol).

In your report, include the verification plan and answer the following question:

- How did you choose the verification priority for each requirement?

6.2 A2. Integration of *uVC*s in the testbench

Lecture references:

1. Lecture03, LE05, E04, LE06, LE07, E05
2. Lecture04, LE08, E06, LE09, E07

The following *uVC*s are provided in `assignment/marb/src/tb/uvc`:

- A `APB` *uVC*
- A `SDT` *uVC*
- A `SSDT` *uVC*

The `APB` *uVC* is already fully integrated into the provided testbench skeleton. Generally for this assignment the `APB` *uVC* you only need to interacted with through the provided read and write functions in the virtual sequencer. You should still try and get an understanding of how the *uVC* works and how it is integrated into the testbench.

In this task you need to integrate the *SDT* and `SSDT` *uVC*s in the testbench and create a minimal test case(s). The minimal testcase(s) should run both a read and a write transaction and configure the *Memory Arbiter* in both the dynamic and static mode.

Before you start integrating a *uVC* you take a look at the *uVC* source code. At minimum you should look at the *uVC* specific configuration object, as well as the sequence library and try to answer the following questions:

- What configuration options are there, what do the do, do they need to be changed from their default values?
- What sequences does the *uVC* provide?

You can refer to *E07: PyUVM uVC integration* or the steps below to get started. *HINT:* Step 1 and 2 are already partly present in the testbench.

1. Create a testbench configuration object and instantiate a configuration object for each *uVC*;
2. In the base test instantiate the testbench configuration object and pass it to the environment through the `ConfigDB`;
3. Create and connect the interface in the base test;
4. Create the agent in the environment and pass its configuration object through the `ConfigDB`;
5. Connect the agent sequencer to the virtual sequencer;

In your report, include:

- An updated testbench diagram based on `assignment/testbench.drawio`
- Plots of the waveforms generated by the minimal testcase. The waveform plots should show read and write transactions and both static and dynamic priority operation.

6.3 A3. Coverage

Lecture references:

1. Lecture06, LE11, E09

The *SDT* and *sSDT uVC*s both provide covergroups. Run your testcase from *A2* and analyze the coverage generated by the built-in *uVC* covergroups.

In your report show the collected coverage labeled as *"A3: Basic UVC coverage"*. Discuss the following questions:

- Is the coverage provided by the *uVC*s sufficient for the verification plan? Why or why not?
- Which additional coverpoints are needed?

Then implement functional coverage for the register values. The covergroup should contain the following coverpoints:

- `cp_enable` to cover the `enable` bit in the control register.
- `cp_mode` to cover the `mode` bits in the control register. Only sampled when enable==1
- `cp_prio` to cover the priority register. Only sampled when enable==1 and mode==`dynamic`.
- A cross of `cp_enable` and `cp_mode`.

Instantiate the covergroup in the base sequence and sample the coverage once per register configuration within the sequence after the stimuli are ran. See below for a possible example:

```
# Within the virtual sequence

# Configure the MARB
self.sequencer.write_ctrl_reg(
  enabled = True,
  mode = MarbMode.STATIC,
)

# Run some stimuli
cl_sdt_single_zd_seq.start(self.sdt_sequencer)

# Sample coverage
self.register_covergroup.sample(
  self.sequencer.register_model.enable,
  self.sequencer.register_model.mode,
  self.sequencer.register_model.prio0,
  self.sequencer.register_model.prio1,
  self.sequencer.register_model.prio2
)
```

Run the testcase again and report the collected coverage labeled as *"A3: Basic register coverage"* in your report. Discuss the following questions:

- Why is it important to sample the coverage after the stimuli are run?
- What additional testcases are needed to close the coverage?
- Is it possible to close the cross coverage?

6.4 A4. Fully random testcase

Lecture references:

1. Lecture05, LE10, E08

In this task you need create a fully random testcase. Before you start discuss in your group what aspects of the test can and should be randomized. Make sure to use `pyvsc` such that you can add constraints to the random variables later on.

Run the testcase and report the coverage labeled as *"A4 Coverage"*. Experiment with the number of transactions send.

In your report, discuss the following questions:

- Which aspects of the testbench can and should be randomized?
- How can timing, sequence items, priorities, and other randomized values be constrained?
- Which coverpoints are hit by the randomized test cases, and why?
- Which coverpoints are not hit by the randomized test cases, and why?
- What changes when you increase the number of transactions?
- Which additional tests would be required to close coverage? You do not need to implement these tests.

6.5 A5. *sSDT* and *SDT* Protocol Checkers

Lecture references:

1. Lecture07, LE12, E10

The *sSDT uVC* provides interface assertions in `ssdt_if_assertions.py`. The *SDT* interface assertions are missing. Implement checkers for the *SDT* protocol according to the provided protocol specifications and requirements.

Then integrate both the *sSDT* and *SDT* checkers into your testbench.

In your report, discuss the following question:

- How does the checker design translate the *SDT* protocol requirements into assertions?

6.6 A6. Reference Model and Scoreboard

Lecture references:

1. Lecture07, LE13, E11

The scoreboard and reference model both operate on the TLM level. This means, the timing aspect of transaction is abstracted away (See LE05 CRV Intro, Transaction Slides). Because of this abstraction level, we have no reliable way of checking the arbitration based on the transactions broadcasted by the analysis ports of the producer and consumer *uVC*s. You can check this yourself by comparing the time stamps of the producer sequence *SDT* sequence item being send via the analysis port with the waveform. You will see, that the sequence item is only broadcast *after* the CIF has received an `ack`. At this point the arbitration has already happened. Checking the correctness of the arbitration needs to be done in a pin-level assertion and is the content of task A7.

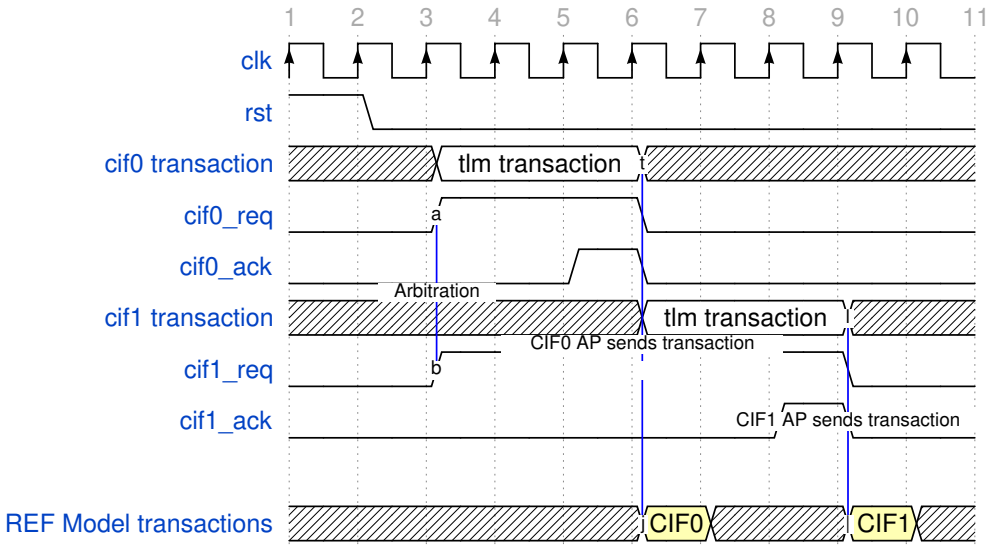


Fig. 6.1: Timing of TLM transactions and arbitration

Consequently the reference model should only mirror the functional transformation of a transaction, which in case of the *Memory Arbiter* is the calculation of the CRC8 checksum.

Since the checksum is not part of the *SDT* transaction, we need another sequence item type: `cl_marb_seq_item`, which extends `cl_sdt_seq_item` and adds an additional field for the checksum value. `cl_marb_seq_item` also needs a custom `__eq__` function, analogue to `cl_sdt_seq_item`.

The reference model receives a `cl_sdt_seq_item` and transform it into a `cl_marb_seq_item`, calculating the checksum based on the `data` and `addr` field.

To connect the consumer *uVC*s with the scoreboard a MIF-CRC connector component as seen in Figure 5.1, which combines the *SDT* sequence item of the MIF and the *sSDT* sequence item of the CRC interface into a `cl_marb_seq_item`.

The MIF-CRC-connector should extend `uvm_component` needs to have:

- A `uvm_tlm_analysis_fifo` to connect to the MIF analysis port.
- A `uvm_tlm_analysis_fifo` to connect to the CRC Interface analysis port.
- A `uvm_analysis_port` to broadcast the combined transaction.

A skeleton for the scoreboard is already given in `assignment/marb/src/tb/cl_marb_scb.py`,

only missing the comparison step.

To implement and integrate the reference model and scoreboard you need to:

- Create the reference model component.
- Create the MIF-CRC-connector.
- Create the extended `cl_marb_seq_item`.
- Add the comparison step in the scoreboard.
- Instantiate and connect all components as seen in Figure 5.1.

In your report, replace the testbench diagram from A2 with an updated diagram that includes the Reference Model and Scoreboard and their connections. Answer the following questions:

- What does the Reference Model do?
- What does the Reference Model not do?

6.7 A7. *Memory Arbiter* Timing Checker

Lecture references:

1. Lecture07, LE12, E07

Since the arbitration of the *Memory Arbiter* has only a temporal effect on the transactions. We can not check it using a reference model on *TLM* level. Instead we need to make use of an assertion, which checks the correctness of the arbitration.

The *Memory Arbiter* timing checker needs a handle to the three CIFs and the MIF. The checker also needs to know the current register configuration. The read/write functions in the virtual sequencer mirror the register status into the global `cl_marb_reg_model` object. The handle of the register model can be passed to the timing checkers in the connect phase analogue to how its passed to the virtual sequencer.

In your report document your checker design and answer the following question:

- How does the checker design translate the *Memory Arbiter* requirements into assertions?

6.8 A8. Additional Tasks

The following tasks are optional, see below for more detailed descriptions on each task. If you choose to do these tasks then you must complete them in order. You cannot pick them in random order.

1. Implement test cases to close coverage.
2. Create a multiple acknowledge checker.
3. Create test cases to assert all checkers.

4. Refactor your testbench to support parametrization with the DUT parameters `DATA_WIDTH` and `ADDR_WIDTH`.
5. Add coverage for the parameters.

1. Coverage Closure

In your report discuss what coverage is still missing after A4 and what additional tests are needed to close coverage. Implement the missing tests. Report the closed coverage labeled *"A8: Closed Coverage"*.

2. Multiple ACK checker

Implement a checker component that will monitor the ack signals of each CIF in order to check that only a single ACK is detected during an operation. Two (or more) CIFs cannot be ACK'ed in the same clock cycle.

3. Additional testcases for checkers

In your report discuss the following questions:

- How did you verify that the checkers and assertions are correct?
- Are all assertions exercised by the test cases?

If not all assertions are exercised, implement the missing testcases.

4. Parametrization

The *Memory Arbiter* can be parametrized with `ADDR_WIDTH` and `DATA_WIDTH`. The parameters are available in the test as `cocotb.top.ADDR_WIDTH` and `cocotb.top.DATA_WIDTH` respectively. Refactor the testbench to support different parameters from the default.

5. Parameter coverage

Implement coverage for DUT the parameters.